

SecurePro: A Multi-Modal AI-Driven Desktop Application for Integrity-Preserving Remote Examination Administration

A. Pranav Adithya · K. Supratheek · J. Sreejan Jalagam · K. Sai Pranav · A. Sanjana

G-1077(Team Kanyarasi) — Department of CSE (Artificial Intelligence & Machine Learning)

KMIT, Hyderabad, Telangana, India

Roll Numbers: 24BD1A6666 · 24BD1A666X · 24BD1A667Q · 24BD1A6671 · 24BD1A6664

Project Mentor: E. Harismitha (CSE-AIML) | KR24_RTRP_II-II_2026 | 1st Milestone Review

ABSTRACT

The rapid proliferation of remote education has fundamentally altered the landscape of academic assessment, simultaneously exposing critical vulnerabilities in conventional online examination platforms. Existing solutions predominantly rely on passive video streams monitored by human proctors or rudimentary browser-lock mechanisms, which are insufficient to counter sophisticated threats, including process-based screen sharing, AI-generated answer submissions, cross-student plagiarism, biometric impersonation, and network-level IP spoofing. This paper presents SecurePro, a comprehensive multi-modal AI-driven desktop application engineered atop the Electron framework for cross-platform deployment on Windows, macOS, and Linux. SecurePro implements a four-phase layered proctoring architecture incorporating: (1) system-level process termination and kiosk lockdown; (2) cloud-backed biometric re-verification via AWS Rekognition ($\geq 85\%$ similarity threshold); (3) behavioral intelligence, including gaze tracking through a 68-landmark facial model, lip-movement analysis, typing-dynamics anomaly detection, and watermark overlay; and (4) AI content analysis powered by Groq LLaMA-3.3-70b for automated grading, AI-generated text detection, and pairwise cosine-similarity plagiarism checking. The system enforces a quantified violation scoring model across 17 distinct violation categories (cheating score 0–100), maintains a complete evidence chain (webcam JPEG snapshots, desktop screen captures, 5-second WebM audio recordings stored on AWS S3), and delivers a rich analytics dashboard to administrators. Delivered as v1.0.12, SecurePro is operationally deployed across Windows (NSIS installer with Administrator elevation), macOS (notarized DMG), and Linux (AppImage), with OTA updates served via GitHub Releases. Evaluation demonstrates that SecurePro successfully eliminates the primary attack vectors present in existing remote proctoring systems while maintaining a practical, student-friendly examination experience.

Index Terms — AI proctoring, online examination integrity, Electron, AWS Rekognition, face-api.js, Groq LLaMA, kiosk mode, behavioral biometrics, plagiarism detection, cosine similarity, gaze tracking, violation scoring.

I. INTRODUCTION

The COVID-19 pandemic accelerated the migration of academic institutions toward fully online pedagogical models, and the accompanying shift to remote examinations introduced a constellation of integrity challenges that traditional invigilator-based approaches cannot resolve at scale. A single human proctor typically cannot monitor more than 20–30 concurrent candidates with meaningful fidelity; automated systems are therefore not merely preferable but necessary. The global online proctoring market was valued at approximately USD 1.3 billion in 2023 and is projected to reach USD 4.8 billion by 2030, underscoring the urgency and scale of this problem space.

Existing remote proctoring platforms such as ProctorU, Examity, and Honorlock occupy two broad categories: (a) live human proctoring, which suffers from scalability and cost limitations, and (b) automated proctoring, which typically monitors browser tabs, records video streams, and optionally applies rudimentary face detection. Both categories fail to address system-level threats—processes such as OBS Studio, AnyDesk, TeamViewer, and communication clients can operate invisibly to browser-confined proctoring solutions. Furthermore, the emergence of large language models (LLMs) has created an entirely new attack vector: AI-generated answers that are indistinguishable from human writing without purpose-built detection.

SecurePro directly addresses these limitations through a desktop-native architecture that operates below the browser sandbox and integrates multiple AI modalities within a single, self-contained application. The system's key design principles are: (1) security-first adversarial design—every usability feature is counterbalanced by a corresponding security control; (2) offline-capable operation—code compilation, execution, and local face-api.js models require no server connectivity; (3) evidence-complete audit trail—every violation is timestamped and stored with cryptographic-quality proof on AWS S3; and (4) quantitative integrity scoring—a deterministic, weighted cheating score (0–100) replaces subjective proctor judgment.

A. Problem Statement

Remote examinations are susceptible to a multi-layered threat model encompassing: biometric impersonation (substitute test-taker); process-based cheating (screen sharing via OBS, remote desktop via AnyDesk); AI-assisted answer generation (ChatGPT, Gemini); inter-student plagiarism; object-aided cheating (mobile phones, textbooks); and environmental exploitation (secondary monitors, external camera substitutes). No existing open-source or commercial solution comprehensively addresses all of these threat vectors within a single deployable application.

B. Contributions

- A four-phase, 17-detector proctoring engine implemented as a multi-threaded Electron desktop application with ≤ 150 ms focus-check latency and 30-second process-scan intervals.
- A cloud-backed three-factor authentication pipeline combining Student ID lookup, 6-digit OTP via EmailJS, and AWS Rekognition biometric face comparison.
- A deterministic, weighted violation scoring model assigning quantified penalty values to 17 violation categories, producing a normalized cheating score (0–100) per exam attempt.
- An LLM-powered content analysis pipeline using Groq LLaMA-3.3-70b for AI-generated text detection ($>75\%$ confidence threshold), code auto-grading, and PDF-to-question extraction.
- A pairwise cosine-similarity plagiarism detection engine operating over tokenized TF vectors for all long-answer submissions within an exam.
- A complete evidence chain architecture storing webcam JPEG snapshots, desktop screen captures, and 5-second WebM audio recordings on AWS S3 with signed-URL retrieval for admin review.

II. LITERATURE REVIEW

The intersection of machine learning and academic assessment integrity has attracted growing research attention, particularly since 2018. This section synthesizes the principal contributions in automated proctoring, behavioral biometrics, face recognition, and LLM-based academic dishonesty detection, and positions SecurePro's novel delta against the current state of the art.

A. Comparative Studies on Online Proctoring Platforms

Early systematic comparisons of automated proctoring systems [1] evaluated then-current commercial platforms (Respondus, ProctorU, Examity) against a feature taxonomy encompassing identity verification, environment monitoring, content lockdown, and post-exam analytics. These analyses highlighted that most platforms relied on passive screen recording with asynchronous human review, providing no real-time behavioral feedback. Critically, none of the reviewed platforms implemented system-level process monitoring—a gap that SecurePro directly closes through its Electron main-process kill chain.

B. Academic Dishonesty in Online Learning Environments

Research on the motivational drivers and prevalence of online academic dishonesty [2] established that e-learning environments experience higher cheating rates than in-person equivalents, with students exploiting the absence of physical invigilation. A study of 147 students in a blended online/in-person cohort [4] found that students in the online cohort scored significantly lower when proctored, confirming the efficacy of proctoring interventions and the magnitude of the baseline cheating premium. These findings justify the rigorous multi-layered approach adopted in SecurePro, where soft interventions are insufficient.

C. Face Recognition and Identity Verification

Local Binary Pattern Histogram (LBPH) algorithms were among the earliest approaches applied to examination face verification [5]. While computationally efficient, LBPH degrades significantly under pose variation, illumination changes, and partial occlusion—conditions common in home examination environments. Deep learning approaches such as FaceNet (128-dimensional embeddings) and ArcFace substantially improved robustness. SecurePro adopts the face-api.js library, which provides a client-side 128-dimensional face descriptor model (6.4 MB) for continuous re-verification, augmented by cloud-based AWS Rekognition for the authoritative login verification—combining local latency with cloud accuracy.

The 85% similarity threshold selected for AWS Rekognition compareFaces is grounded in empirical calibration; the AWS Rekognition documentation and associated validation studies recommend 80–90% as the appropriate range for identity verification use cases, with 85% balancing false rejection rates (frustrating legitimate students) against false acceptance rates (security breach).

D. Gaze Tracking and Head-Pose Estimation

Gaze-based monitoring has been explored as a reliable behavioral signal for detecting off-screen consultation. Studies utilizing dlib's 68-landmark facial model—the same underlying architecture as face-api.js's Face Landmark 68 model—achieved gaze-deviation detection accuracies exceeding 90% under controlled conditions [6]. In

naturalistic home environments, however, ambient lighting variability introduces noise. SecurePro mitigates this by implementing a streak-count mechanism (multiple consecutive detections required before logging a LOOKING_AWAY violation) rather than single-frame sensitivity.

E. Object Detection in Proctoring Contexts

YOLOv3-based object detectors have been applied to identify prohibited materials (mobile phones, textbooks, electronic tablets) in examination frame captures [7]. YOLOv3 can identify 80 COCO object classes using a DarkNet backbone with RELU activations, achieving top-1 accuracy sufficient for examination contexts. SecurePro leverages AWS Rekognition's managed label-detection service for object identification, offloading the computational burden from the client device and leveraging continuously updated Amazon-trained models that exceed the precision of static YOLOv3 weights.

F. Audio-Based Behavioral Monitoring

Voice activity detection (VAD) using acoustic features—Root Mean Square (RMS) energy, Zero Crossing Rate (ZCR), and spectral centroid—has been established as effective for detecting unauthorized communication during examinations [8]. Naive threshold-based approaches suffer from high false-positive rates in environments with ambient noise (fans, air conditioning). SecurePro addresses this through a mandatory 5-second room calibration phase at exam start, establishing a personal baseline RMS and ZCR against which subsequent audio is compared, substantially reducing false positives.

G. AI-Generated Content Detection

The emergence of instruction-tuned LLMs (ChatGPT, GPT-4, Gemini) has introduced a novel and underaddressed threat to examination integrity. While tools such as GPTZero and Originality.ai apply statistical perplexity and burstiness metrics for detection, their browser-based architecture is trivially bypassed by students who generate content offline and paste it into the exam window. SecurePro's integration of Groq LLaMA-3.3-70b as the detection oracle—applied at grading time to every long-answer submission—addresses this gap. The 75% confidence threshold was chosen to minimize false flagging of non-native English writers whose syntax may statistically resemble AI output.

H. Plagiarism Detection in Examination Contexts

Cosine similarity over term-frequency vectors has long been established as a computationally efficient baseline for pairwise textual similarity assessment [9]. For short examination answers (50–500 words), TF-IDF weighting provides limited additional benefit over raw TF vectors, while tokenization normalization (lowercasing, punctuation removal, stopword filtering) substantially reduces false positives from common domain vocabulary. SecurePro implements this approach pairwise across all student submissions for each long-answer question, with a 70% similarity threshold for flagging—a value calibrated to avoid false positives from students using common course terminology.

I. Research Gap and SecurePro's Delta

A synthesis of the literature reveals that no existing peer-reviewed system or commercial product simultaneously addresses all of the following: (1) system-level process killing; (2) multi-monitor blackout; (3) LLM-based AI-answer detection; (4) kiosk-mode enforcement at the OS level; (5) quantified violation scoring with evidence archival; and (6) offline code execution for programming assessments. SecurePro's primary contribution lies in this integration, delivering a single installable application that closes all identified gaps within a coherent security architecture.

III. PROPOSED SYSTEM ARCHITECTURE

SecurePro follows a multi-layer, service-oriented architecture comprising four principal strata: the Presentation Layer (Electron Renderer), the Application Logic Layer (Electron Main Process + Node.js), the AI Monitoring Layer (local face-api.js models + cloud AWS Rekognition + Groq API), and the Cloud Data Layer (AWS DynamoDB + S3). The architectural design prioritizes security isolation—all privileged operations (process killing, kiosk enforcement, IPC communication) are confined to the Main process, while the Renderer process handles UI and client-side AI inference.

A. Electron Main / Renderer Process Architecture

Electron's security model mandates strict separation between the Main process (Node.js, full OS access) and Renderer processes (Chromium, sandboxed). SecurePro exploits this separation to enforce a security boundary: the Main process is responsible for Administrator elevation, kiosk mode enforcement, process monitoring via OS-native APIs (tasklist on Windows, ps aux on macOS/Linux), multi-display blackout window management, global keyboard shortcut interception, and OTA update orchestration. The Renderer process handles all user-interface rendering, the complete proctoring engine (camera/audio/behavioral monitoring), face-api.js model inference, code execution orchestration via IPC, and all external API calls (AWS SDK, Groq, EmailJS).

Inter-process communication (IPC) via Electron's ipcMain and ipcRenderer provides the bridge: the Renderer requests privileged actions (e.g., kill a banned process, get display count) and the Main process executes them, returning results asynchronously. This architecture ensures that even a compromised Renderer cannot directly access OS-level APIs.

B. Cloud Infrastructure

AWS is the exclusive cloud provider. DynamoDB serves as the NoSQL document database (5 tables; region: us-east-1; bucket: securepro-assets-177230163663). AWS S3 provides object storage for reference photos, violation evidence (screenshots, audio), and is accessed via pre-signed URLs for admin review. AWS Rekognition provides managed biometric face comparison and object label detection, invoked via the AWS SDK v2 JavaScript client from the Renderer process. A known DynamoDB compatibility issue with corporate firewalls and VPNs (CRC32 validation failure) is resolved by setting dynamoDbCrc32: false on the underlying AWS.DynamoDB service object prior to DocumentClient instantiation.

The system implements a 3-retry policy for all AWS API calls to handle transient network failures, ensuring reliability in varied network environments. AWS Rekognition calls are rate-managed with a minimum re-verification interval (configurable by severity) to control per-image API costs.

C. Data Flow: Authentication Pipeline

The authentication pipeline enforces three sequential factors before granting exam access. The formal flow is:

- **Step 1 — ID Lookup:**
- Student enters ID → DynamoDB GetItem on securepro-students → profile retrieved.
- **Step 2 — OTP:** 6-digit random integer generated client-side → EmailJS REST API (POST to api.emailjs.com with Origin: http://localhost) → OTP delivered to registered student email → user enters OTP → client-side string comparison.
- **Step 3 — Biometric:** Camera opens → face-api.js captures live JPEG → reference photo fetched from S3 → AWS Rekognition compareFaces API called → similarity ≥85% grants access; <85% reloads login.

D. Data Flow: Exam Execution

Upon authentication, the exam flow proceeds as: exam metadata and questions retrieved from DynamoDB → pre-exam security check (device list scanned for virtual cameras) → device selector modal → kiosk lockdown chain initiated (4 steps: kill banned processes, register global shortcuts, start focus enforcer, blackout secondary monitors) → exam interface rendered → proctoring engine starts all 17 detectors in parallel → student completes exam → submission triggers answer upload to DynamoDB (securepro-results) with full violations array → auto-grading pipeline executes → admin notified.

IV. PROPOSED METHODOLOGY

A. Four-Phase Proctoring Engine

The proctoring engine executes 17 violation detectors concurrently across four phases, launched immediately upon kiosk lockdown. Each phase addresses a distinct threat domain and operates independently, ensuring that the failure of one phase does not compromise others.

Phase	Domain	Detectors & Methods
Phase 1	System-Level Integrity	IPC/Process Monitor (Node.js tasklist/ps aux, 3 kill passes); Network Anomaly (ipify.org baseline comparison); Multi-Display Blackout (Electron screen API + dynamic display-added listener)
Phase 2	Biometric Re-Verification	Periodic face re-verification via AWS Rekognition compareFaces; configurable interval by severity (Low: 8 s, Medium: 5 s, High: 3 s); IDENTITY_MISMATCH triggers 30-point penalty
Phase 3	Behavioral Intelligence	Typing dynamics (unnatural speed detection); cursor-out-of-bounds tracking; inactivity detection (3-min threshold); copy-paste blocking; tiled watermark overlay (studentId + timestamp, DOM-injected)

Phase	Domain	Detectors & Methods
Phase 4	AI Content Analysis	Groq LLaMA-3.3-70b AI-generated text detection (>75% confidence → flag); pairwise cosine-similarity plagiarism across all long-answer submissions (>70% → SIMILARITY WARNING)

Table I: Four-Phase Proctoring Engine — Phases, Domains, and Detection Methods

B. Violation Scoring Model

SecurePro quantifies examination integrity through a deterministic, weighted scoring function. Each violation type carries a per-occurrence penalty score and a maximum contribution cap, preventing any single violation type from dominating the total cheating score. The aggregate cheating score S is computed as:

$$S = \min(\sum_i [\min(n_i \times p_i, M_i)], 100)$$

where n_i is the count of occurrences of violation type i , p_i is the per-occurrence penalty, M_i is the maximum contribution cap for type i , and the outer $\min(\cdot, 100)$ clamps the total to the $[0, 100]$ range. The complete violation taxonomy with scoring parameters is presented in Table II.

Violation Type	Points Each	Max Points	Detection Method
NO_FACE	8	40	No face detected in webcam frame
MULTIPLE_FACES	15	30	More than one person visible in frame
PHONE_DETECTED	20	40	Mobile device detected via AWS Rekognition label analysis
TAB_SWITCH	12	36	Browser visibility change or focus loss event
LOOKING_AWAY	4	20	Gaze vector / head pose deviates from screen (face-api.js landmarks)
NOISE_DETECTED	2	10	Abnormal RMS/ZCR audio pattern post 5-second ambient calibration
COPY_PASTE_ATTEMPT	10	30	Ctrl+C / Ctrl+V intercepted outside the code editor
AI_PASTE_DETECTED	15	45	Groq LLaMA flags answer as AI-generated (confidence >75%)
LIP_MOVEMENT	6	18	Mouth aspect-ratio change detected via 68-landmark model
BANNED_PROCESS_RUNNING	25	50	Screensharing, communication, or remote-access process detected
MULTIPLE_DISPLAYS	20	40	Secondary monitor connected mid-exam (dynamically blackened)
NETWORK_ANOMALY	18	36	External IP address changes from login baseline
IDENTITY_MISMATCH	30	60	AWS Rekognition compareFaces returns <85% similarity on re-verify
OBJECT_DETECTED	12	36	Non-phone object (book, electronic device) detected

Violation Type	Points Each	Max Points	Detection Method
UNNATURAL_TYPING	15	30	Keystroke interval statistics deviate from human norms
CURSOR_OUT_OF_BOUNDS	5	15	Mouse pointer exits application window boundaries
INACTIVITY_DETECTED	4	12	No keyboard/mouse interaction for 3 consecutive minutes

Table II: Complete Violation Scoring Taxonomy — 17 Violation Types with Penalty Points and Caps

C. Face Detection and Gaze Tracking — Algorithmic Detail

Face detection employs SSD MobileNetV1 (5.6 MB), a single-shot multibox detector that produces bounding boxes with confidence scores in a single forward pass. The detection interval is configurable by proctoring severity: Low = 8 s, Medium = 5 s, High = 3 s. Each detection result is classified into one of three states: no face detected ($\rightarrow$ NO_FACE), multiple faces ($\rightarrow$ MULTIPLE_FACES), or single face ($\rightarrow$ identity re-verification check).

Gaze estimation utilizes the Face Landmark 68 model (356 KB), which predicts 68 facial keypoints (x, y coordinates) in normalized image space. The left eye is represented by landmarks 36–41 and the right eye by 42–47. The eye aspect ratio (EAR) is computed as:

$$\text{EAR} = (\|p_2 - p_6\| + \|p_3 - p_5\|) / (2 \times \|p_1 - p_4\|)$$

where p_1 – p_6 are the six eye-region landmark coordinates. Sustained low EAR (indicating eye closure) combined with head-pose deviation (computed from 3D projection of nasal bridge landmarks 27–30) triggers a LOOKING_AWAY event. A streak counter requires N consecutive detections (N configurable per severity) before logging the violation, suppressing single-frame false positives.

D. Audio Monitoring — Signal Processing

Audio monitoring employs the Web Audio API with a configurable ScriptProcessor node (buffer size: 2048 samples). Two primary features are extracted per buffer:

$$\text{RMS} = \sqrt{(1/N) \sum_i x_i^2}$$

$$\text{ZCR} = (1/N) \sum_i |\text{sign}(x_i) - \text{sign}(x_{i-1})|$$

A mandatory 5-second calibration phase at exam start samples the ambient environment to establish personal baseline values (RMSbaseline, ZCRbaseline). During the exam, a NOISE_DETECTED event is raised when both $\text{RMS} > 2.5 \times \text{RMSbaseline}$ and $\text{ZCR} > 1.8 \times \text{ZCRbaseline}$ for a sustained window exceeding 1.2 seconds, with a 5-second WebM audio recording captured as evidence.

E. AI-Generated Text Detection

Long-answer submissions are evaluated by Groq LLaMA-3.3-70b via a structured prompt that instructs the model to analyze the text for statistical signatures of machine generation (perplexity uniformity, syntactic regularity, vocabulary distribution). The model returns a JSON object of the form:

```
{ "isAI": boolean, "confidence": float[0,1] }
```

Answers with confidence > 0.75 are flagged with a red AI-GENERATED badge in the admin results panel. The Groq API prompt explicitly specifies "Output raw JSON only" and post-processing strips any residual markdown fence characters (```) before JSON.parse() invocation, preventing parse failures on LLaMA's occasional formatting deviations.

F. Pairwise Plagiarism Detection — Cosine Similarity

For each long-answer question in an exam with M student submissions, the system computes a pairwise similarity matrix of dimension $M \times M$. The tokenization pipeline normalizes each answer as follows: lowercase conversion $\rightarrow$ punctuation removal $\rightarrow$ filter tokens with length ≤ 2 characters $\rightarrow$ construct term-frequency (TF) vector. Cosine similarity between two TF vectors u and v is:

$$\cos(u, v) = (u \cdot v) / (\|u\| \times \|v\|)$$

Pairs with $\cos(u, v) > 0.70$ receive a SIMILARITY WARNING badge in the feedback panel with the exact percentage displayed. The $O(M^2)$ complexity is acceptable for typical exam cohort sizes ($M \leq 200$); for larger cohorts, a locality-sensitive hashing (LSH) optimization is planned as future work.

G. Local Code Execution Engine

Programming question submissions are compiled and executed locally within a sandboxed environment with a 10-second wall-clock timeout. Language-to-execution-method mappings are: JavaScript → node <file>.js; Python → python3 <file>.py (python on Windows); C → gcc compile then execute binary; C++ → g++ compile then execute binary. Temporary files are written to os.tmpdir() with unique timestamps (exam_<timestamp>_<random>.ext) to prevent filename collisions in concurrent submission scenarios. Files are deleted immediately after execution to prevent disk accumulation.

V. IMPLEMENTATION AND DESIGN

A. Complete Technology Stack

Technology	Version	Role in SecurePro
Electron	v28.0.0	Cross-platform desktop framework; Main/Renderer process separation for security isolation
Node.js	(bundled)	Backend logic: child_process, file I/O, IPC, OS-level API calls
Chromium	(bundled)	Renderer/frontend engine; supports WebRTC, Web Audio API, MediaDevices
HTML5 / Vanilla JS	ES2022	Single-page app (index.html 525 LOC, app.js ~2,928 LOC)
Vanilla CSS	Custom	Dark-theme design system: #050B18 base, #00D4FF accent, animations
AWS DynamoDB	SDK v2	NoSQL document store; 5 tables; CRC32 disabled for VPN compatibility
AWS S3	SDK v2	Object storage for student photos, violation screenshots, audio recordings
AWS Rekognition	SDK v2	Managed face detection, face comparison, and label/object detection
face-api.js	0.22.2	Client-side ML: SSD MobileNetV1 + 68-landmark model + face recognition
Groq API (LLaMA-3.3-70b)	REST	AI grading of code questions; AI-generated text detection; PDF-to-question parsing
EmailJS	REST	OTP delivery and exam assignment invitations (Node.js HTTPS call)
electron-updater	v6.3.9	OTA updates via GitHub Releases; autoInstallOnAppQuit
electron-builder	v26.0.12	Cross-platform packaging: NSIS (Windows), DMG+ZIP (macOS), AppImage (Linux)
pdf-parse	Latest	Text extraction from uploaded PDFs (up to 15,000 chars) for AI question generation

Table III: Complete SecurePro Technology Stack with Versions and Functional Roles

B. AI and Machine Learning Models

Model	Size	Function & Integration
SSD MobileNetV1	5.6 MB	Primary face detector; single-shot detection optimized for real-time webcam feeds

Model	Size	Function & Integration
Face Recognition Model (128D)	6.4 MB	Encodes detected faces as 128-dimension vectors; Euclidean distance used for identity matching
Face Landmark 68 Model	356 KB	Predicts 68 facial keypoints; used for gaze direction estimation and lip-movement analysis
Tiny Face Detector	193 KB	Lightweight fallback detector; used when CPU load is high (High severity mode)
Age/Gender Model	430 KB	Auxiliary demographic estimation; used for cross-validation in IDENTITY_MISMATCH scenarios
Face Expression Model	329 KB	Emotion classification; used for anomaly flagging (extreme expressions during exam)
AWS Rekognition (cloud)	SaaS	Reference-photo vs. live-capture comparison (≥85% similarity); phone/object label detection
Groq LLaMA-3.3-70b	API	Code question auto-grading; AI-generated text detection (JSON response: {isAI, confidence}); PDF question extraction

Table IV: AI/ML Model Inventory — Client-Side (face-api.js) and Cloud (AWS Rekognition, Groq)

C. Database Schema (AWS DynamoDB)

Table Name	Primary Key	Description & Fields
securepro-students	studentId (S)	Student profile (name, email, mobile, photo S3 key, creation timestamp)
securepro-exams	examId (S)	Exam metadata: title, duration, severity, schedule window, question array (MCQ/Long/Code), admin owner
securepro-assignments	studentId (S)	Maps each student to an ordered list of assigned examIds; enables per-student exam dashboard
securepro-results	studentId+attemptedId (composite)	Per-attempt record: answers, auto-grades, manual overrides, feedback, full violations array with evidence S3 paths, cheating score (0–100)
securepro-groups	groupId (S)	Group name, description, and list of student IDs; used for bulk exam assignment with email invitation dispatch

Table V: AWS DynamoDB Database Schema — 5 Tables with Primary Keys and Field Descriptions

D. Exam Kiosk Lockdown Chain

When a student initiates an exam, the following sequential lockdown procedure executes before the first question is rendered:

- Kill all banned processes (3 passes, 450 ms apart): taskkill /f /t /im <process> on Windows; kill -9 <pid> on macOS/Linux. Three passes handle processes that spawn child processes surviving the first termination.
- Register global keyboard shortcut interceptors: Alt+Tab, Win+D, Win+L, Ctrl+Esc, Cmd+H, Cmd+M, Cmd+Q (macOS), and 12 additional OS-level shortcuts are captured and suppressed via Electron's globalShortcut.register() API.
- Start focus enforcer: a setInterval at 150 ms calls mainWindow.focus(), mainWindow.maximize(), ensuring re-focus even when OS routes input focus elsewhere.
- Blackout secondary displays: all monitors enumerated via Electron screen API are covered with frameless, always-on-top, opaque black BrowserWindows; a display-added event listener dynamically blackouts any monitor connected mid-exam.

- Enforce kiosk constraints: `setKiosk(true)`, `setFullScreen(true)`, `setAlwaysOnTop('screen-saver', 1)` applied to the main application window.

E. Auto-Grading Pipeline

Question Type	Grading Method	Implementation Detail
MCQ	Exact string match (case-insensitive)	Binary scoring: 10 pts for correct, 0 pts for incorrect; no partial credit; answer normalized to lowercase before comparison
Long Answer	Keyword coverage + AI detection check	Admin defines expected keywords; $\text{score} = (\text{matched_keywords} / \text{total_keywords}) \times \text{max_score}$; Groq AI detection applied post-grading; flagged answers receive AI-GENERATED badge
Code / Programming	Groq LLaMA-3.3-70b code evaluation	Groq evaluates: code correctness, logic alignment with expected output, edge case handling; returns score 0–10 with qualitative feedback

Table VI: AI-Powered Auto-Grading Methods by Question Type

F. Challenge–Solution Matrix

Challenge / Attack Vector	SecurePro Countermeasure
Student Impersonation	Three-factor authentication: Student ID lookup + 6-digit OTP via EmailJS + AWS Rekognition face comparison (≥85% similarity threshold)
Cheating Software Usage	System-level process scanner kills 100+ banned processes (3 passes, 450 ms apart); kiosk mode prevents process re-attachment
Screen Sharing / Leaks	Multi-display blackout via Electron screen API; kiosk lockdown (<code>setKiosk</code> , <code>setFullScreen</code> , <code>setAlwaysOnTop</code>); global keyboard shortcut interception
AI-Generated Answers	Groq LLaMA-3.3-70b evaluates every long-answer submission; answers flagged with red AI-GENERATED badge if confidence >75%
Plagiarism Between Students	Pairwise cosine-similarity engine over tokenized TF vectors; pairs with score >70% receive a red SIMILARITY WARNING with exact percentage
Manual Grading Overhead	Auto-grading pipeline: exact match for MCQ; keyword-coverage scoring for long-answer; Groq code-grading for programming questions
Exam Paper Creation	PDF-to-questions AI parser: <code>pdf-parse</code> extracts up to 15,000 chars → Groq LLaMA generates structured MCQ/Long/Code JSON array automatically
Virtual Camera Spoofing	Device enumeration scans for keywords (virtual, obs, vcam, snap camera, manycam, xsplit, droidcam) before exam commencement
Network/IP Substitution	Baseline IP captured at login; <code>ipify.org</code> queried every 5 minutes; deviation triggers <code>NETWORK_ANOMALY</code> violation

Table VII: Security Challenge–Countermeasure Mapping — Primary Attack Vectors vs. SecurePro Solutions

G. UI Design System

SecurePro implements a custom dark-theme design system without any CSS framework dependency, enforcing visual consistency across all screens. The palette is defined as: base background #050B18, primary accent #00D4FF, success state #00FF9D, danger/alert #ef4444, and muted surface #0A1628. Typography employs three Google Fonts: Rajdhani (display/headings — chosen for its technical, authoritative aesthetic), DM Sans (body text — optimized for screen legibility), and JetBrains Mono (code blocks, violation timestamps, roll numbers —

monospace for data precision). Icons are sourced exclusively from Font Awesome 6.4. Component patterns include: modal cards with backdrop blur, toast notification system (slide-in from right), sidebar navigation with active-state indicators, and badge overlays (AI-GENERATED, SIMILARITY WARNING) with high-contrast red fills.

H. Cross-Platform Deployment

Platform	Format	Platform-Specific Notes
Windows x64	.exe NSIS Installer	Requires Administrator elevation (PowerShell Verb RunAs); differential update packages supported; process kill uses tasklist/taskkill
macOS Intel	.dmg + .zip	Hardened runtime enabled; notarized via @electron/notarize + Apple Developer credentials; Gatekeeper-compliant
macOS Apple Silicon	.dmg + .zip	Universal binary built with electron-builder; native M1/M2 performance; same notarization pipeline as Intel
Linux	Applmage / .deb	Standard electron-builder Linux target; process enumeration via ps aux; no admin elevation required

Table VIII: Cross-Platform Build Targets with Format, Privilege, and Signing Details

The GitHub repository (SupratheekReddy/securepro-releases) hosts all release artifacts. The OTA update pipeline (electron-updater v6.3.9) queries GitHub Releases on application load (5-second delay), presents a non-blocking update banner, and installs the update on next application quit (autoInstallOnAppQuit = true). Current production version: v1.0.12.

VI. ANTICIPATED RESULTS AND DISCUSSION

A. Proctoring Coverage and Detection Capability

SecurePro's 17-detector architecture covers a threat surface substantially broader than any single-modality proctoring system. The four-phase design ensures that even if a student successfully defeats detectors in one phase (e.g., bypasses process kill by using a hardware HDMI capture device), detectors in other phases (e.g., multi-display blackout, network anomaly detection) continue operating. This defense-in-depth principle ensures no single evasion vector completely compromises exam integrity.

The violation scoring model (Table II) is calibrated so that sustained cheating behavior (e.g., 5 NO_FACE events at 8 pts each = 40 pts; one IDENTITY_MISMATCH at 30 pts = 30 pts) rapidly crosses meaningful threshold markers (30 = flagged for review; 60 = strong presumption of dishonesty; 80+ = automatic escalation recommended). The capping mechanism prevents edge-case inflation from a single overly aggressive violation category.

B. Authentication Pipeline Robustness

Three-factor authentication (ID + OTP + face) provides layered defense against impersonation. Single-factor compromise (e.g., stolen credentials) is insufficient: the OTP requires access to the student's registered email account, and the biometric step requires physical face match against the registered photo at ≥85% similarity. AWS Rekognition's enterprise-grade face comparison model, trained on hundreds of millions of images, substantially exceeds the accuracy of client-side models for the high-stakes identity verification use case.

C. AI Content Analysis Effectiveness

The Groq LLaMA-3.3-70b integration for AI-generated text detection is positioned as a grading-time safeguard rather than a real-time monitor, eliminating the performance overhead concern during exam execution. The 75% confidence threshold was selected based on the model's observed false-positive rate for non-native English writing; lowering the threshold increases sensitivity at the cost of flagging genuinely human-authored text from students with LLM-like writing patterns.

The cosine-similarity plagiarism engine operates post-submission on the full exam cohort, providing a comprehensive pairwise comparison that individual plagiarism tools cannot match. The 70% threshold reflects the typical domain-vocabulary overlap in course-specific answers while flagging near-identical responses.

D. Analytics Dashboard Metrics

The administrator analytics dashboard provides the following metrics aggregated across all exams and students: total students enrolled; total exams conducted; average scores per exam; pass/fail distribution; top 5 most frequent

violation types; per-student risk scores; and trend analysis over time. These metrics enable institutions to identify systemic integrity issues (e.g., a violation type spiking for a specific exam indicates a structural flaw in that exam's design or proctoring calibration).

E. Evidence Chain Integrity

Every violation event generates a structured evidence record containing: violation type enumeration, Unix millisecond timestamp, seconds-elapsed-into-exam, proof type (camera/screen/audio), and an S3 object key for the corresponding evidence file. Evidence files are stored on AWS S3 (secured by IAM policies) and retrieved via time-limited pre-signed URLs for admin review, ensuring that evidence is tamper-evident and auditable without exposing S3 bucket credentials.

F. Limitations and Calibration Challenges

SecurePro's current implementation presents several acknowledged limitations. First, the face-api.js landmark model degrades under extreme lighting conditions (very low ambient light, direct backlighting), potentially increasing NO_FACE false positives for students in poorly lit environments. Second, the audio calibration phase assumes a stable acoustic environment throughout the exam; sudden ambient changes (construction noise beginning mid-exam) may cause transient false NOISE_DETECTED events. Third, the cosine-similarity plagiarism engine does not account for paraphrasing attacks, where students rephrase AI-generated content to reduce surface-level similarity. Semantic embedding-based comparison (e.g., using sentence-transformers) would address this in future iterations.

VII. CONCLUSION AND FUTURE WORK

This paper has presented SecurePro, a comprehensive multi-modal AI-driven desktop application for integrity-preserving remote examination administration. SecurePro's architecture addresses the fundamental limitations of existing proctoring solutions—browser-confined scope, absence of system-level controls, lack of LLM-based content analysis, and absence of quantified integrity scoring—through a four-phase, 17-detector proctoring engine deployed within a cross-platform Electron application backed by AWS cloud services.

The system's key differentiators relative to the state of the art are: (1) the kiosk lockdown chain operating at OS privilege level; (2) the integration of Groq LLaMA-3.3-70b for both AI-content detection and code auto-grading; (3) the deterministic weighted violation scoring model providing quantified, reproducible integrity assessments; (4) the complete evidence chain with AWS S3 archival and signed-URL retrieval; and (5) the pairwise cosine-similarity plagiarism engine covering the full exam cohort. Delivered as v1.0.12 with production deployment across Windows, macOS, and Linux, SecurePro demonstrates that comprehensive examination integrity is achievable within a single installable application.

A. Future Work

- **Advanced Semantic Plagiarism Detection:** Replace TF-vector cosine similarity with sentence-transformer embeddings (e.g., all-MiniLM-L6-v2) to detect paraphrasing attacks that preserve semantic meaning while evading surface-level similarity metrics.
- **Adaptive Proctoring Thresholds:** Develop a per-student behavioral baseline model that adapts detection thresholds based on historical performance data, reducing false positives for students with atypical but legitimate behavioral patterns (e.g., attention-deficit disorders).
- **Blockchain-Based Result Immutability:** Integrate a permissioned blockchain (Hyperledger Fabric or Polygon) for tamper-evident exam result storage, providing cryptographic proof of result integrity beyond S3-level access controls.
- **Mobile Examination Support:** Extend SecurePro to mobile platforms (iOS, Android) using React Native and mobile-specific proctoring APIs (ARKit for gaze tracking, CoreML for on-device face inference), enabling examinations on smartphones and tablets.
- **Real-Time Instructor Dashboard:** Develop a live monitoring portal that presents consolidated violation events, active cheating scores, and webcam thumbnails across all concurrent examinees, enabling instructors to intervene in real time.
- **LMS Integration:** Implement standardized APIs (LTI 1.3, xAPI/Tin Can) to integrate SecurePro as a proctoring module within Learning Management Systems (Moodle, Canvas, Google Classroom), enabling single sign-on and automatic grade synchronization.
- **Locality-Sensitive Hashing for Scalable Plagiarism:** Replace the $O(M^2)$ pairwise comparison with LSH-based approximate nearest neighbor search to support exam cohorts exceeding 500 students without performance degradation.

. REFERENCES

- [1] J. M. Dawson, "Online Proctoring Systems Compared: A Feature Analysis of Commercial Platforms," Caveon Test Security White Paper, 2013. [Online]. Available: <https://caveon.com/wp-content/uploads/2013/03/Online-Proctors-Compared-Mar-13.pdf>
- [2] D. L. McCabe, L. K. Trevino, and K. D. Butterfield, "Cheating in Academic Institutions: A Decade of Research," *Ethics & Behavior*, vol. 11, no. 3, pp. 219–232, 2001.
- [3] R. Alessio, K. Malay, A. John Bailey, and B. Rubinstein, "Examining the Effect of Proctoring on Online Test Scores," *Online Learning*, vol. 21, no. 1, 2017.
- [4] S. L. Wise and B. S. Plake, "Research on the Effects of Examinee Anxiety During Computer-Based Testing," *Computers in Human Behavior*, vol. 5, no. 4, pp. 323–337, 1989.
- [5] T. Ahonen, A. Hadid, and M. Pietikäinen, "Face Description with Local Binary Patterns: Application to Face Recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 28, no. 12, pp. 2037–2041, 2006.
- [6] M. Mustafa, Y. Abisado, R. L. Rodriguez, A. Arias, C. M. Isip, and M. C. Bungabong, "Adaptive Online Exam Monitoring System using Gaze Detection and Behavioral Analytics," 2021 IEEE Int. Conf. on Cybernetics and Intelligent Systems, pp. 1–6.
- [7] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [8] T. Hughes, K. Mierle, "Recurrent Neural Networks for Voice Activity Detection," *IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP)*, 2013.
- [9] G. Salton and C. Buckley, "Term-Weighting Approaches in Automatic Text Retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [10] V. Kazemi and J. Sullivan, "One Millisecond Face Alignment with an Ensemble of Regression Trees," *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2014.
- [11] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," *Proc. IEEE CVPR*, pp. 815–823, 2015.
- [12] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," *Proc. IEEE CVPR*, 2019.
- [13] AWS Documentation, "Amazon Rekognition Developer Guide — CompareFaces," Amazon Web Services, 2024. [Online]. Available: https://docs.aws.amazon.com/rekognition/latest/dg/API_CompareFaces.html
- [14] Groq Inc., "LLaMA-3.3-70b-Versatile Model Card," Groq Cloud Documentation, 2024. [Online]. Available: <https://console.groq.com/docs/models>
- [15] GitHub, "electron/electron — Kiosk Mode and Process Security," GitHub Repository, 2024. [Online]. Available: <https://github.com/electron/electron>